Міністерство освіти і науки України

Національний технічний університет України

Київський політехнічний інститут імені Ігоря Сікорського

Лабораторна робота №4

«Аналітичні SQL-запити (OLAP)»

Роботу виконали
Студенти групи ІО-45, ІО-43
Ренцевич Д. , Гапон А.
Роботу перевірив:
Русінов В.В.

Київ 2026

ЛАБОРАТОРНА РОБОТА № 4

Тема: Аналітичні SQL-запити (OLAP).

Мета: Набути практичних навичок написання аналітичних SQL-запитів (OLAP) для підсумовування тенденцій та створення звітів на базі розробленої схеми бази даних у PostgreSQL. Навчитися обчислювати зведену статистику за допомогою агрегатних функцій, групувати дані (GROUP BY), фільтрувати агреговані результати (HAVING), а також виконувати багатотабличні об'єднання (JOIN).

Завдання:

1.Написати мінімум 4 запити, що містять агрегаційні функції (SUM, AVG, COUNT, MIN, MAX, GROUP BY).

2.Написати мінімум 3 запити, що використовують різні типи джоінів (наприклад, INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN) для об'єднання даних з кількох таблиць.

3.Написати мінімум 3 запити з використанням підзапитів (вибірка з підзапитом у SELECT, WHERE, або HAVING).

4.Виконати створені запити в PostgreSQL (pgAdmin) для власної бази даних та перевірити правильність виводу.

Агрегаційні функції (SUM, AVG, COUNT, MIN, MAX, GROUP BY)

```
-- 1. Підрахунок кількості кімнат за їхнім статусом (вільний, зайнятий, прибирання)
SELECT status, COUNT(*) AS rooms_count
FROM room
GROUP BY status;

-- 2. Загальна сума доходу від усіх проведених платежів
SELECT SUM(amount) AS total_revenue
FROM payment;
```

```sql
-- 3. Середня, мінімальна та максимальна базова вартість типів номерів
SELECT
   AVG(base_price) AS avg_price,
   MIN(base_price) AS min_price,
   MAX(base_price) AS max_price
FROM room_type;

-- 4. Загальна сума платежів по кожному бронюванню з фільтрацією (тільки ті, де сума > 2000)
SELECT booking_id, SUM(amount) AS total_paid
FROM payment
GROUP BY booking_id
HAVING SUM(amount) > 2000;
```

Query Query History

```sql
-- Агрегаційні функції (SUM, AVG, COUNT, M

-- 1. Підрахунок кількості кімнат за їхнім
SELECT status, COUNT(*) AS rooms_count
FROM room
GROUP BY status;
```

Data Output Messages Notifications

	status character varying (20)	rooms_count bigint
1	зайнятий	2
2	прибирання	1
3	вільний	1

```sql
-- 2. Загальна сума доходу від усіх проведених платежів
SELECT SUM(amount) AS total_revenue
FROM payment;

-- 3. Середня, мінімальна та максимальна базова вартість т
```

Data Output Messages Notifications

	total_revenue numeric
1	10000.00

```
-- 3. Середня, мінімальна та максимальна ба
SELECT
    AVG(base_price) AS avg_price,
    MIN(base_price) AS min_price,
    MAX(base_price) AS max_price
FROM room_type;
```

Data Output | Messages | Notifications

	avg_price numeric	min_price numeric	max_price numeric
1	2150.0000000000000000	600.00	4500.00

```
-- 4. Загальна сума платежів по кожно
SELECT booking_id, SUM(amount) AS tot
FROM payment
GROUP BY booking_id
HAVING SUM(amount) > 2000;
```

Data Output | Messages | Notifications

	booking_id integer	total_paid numeric
1	2	5000.00
2	1	4000.00

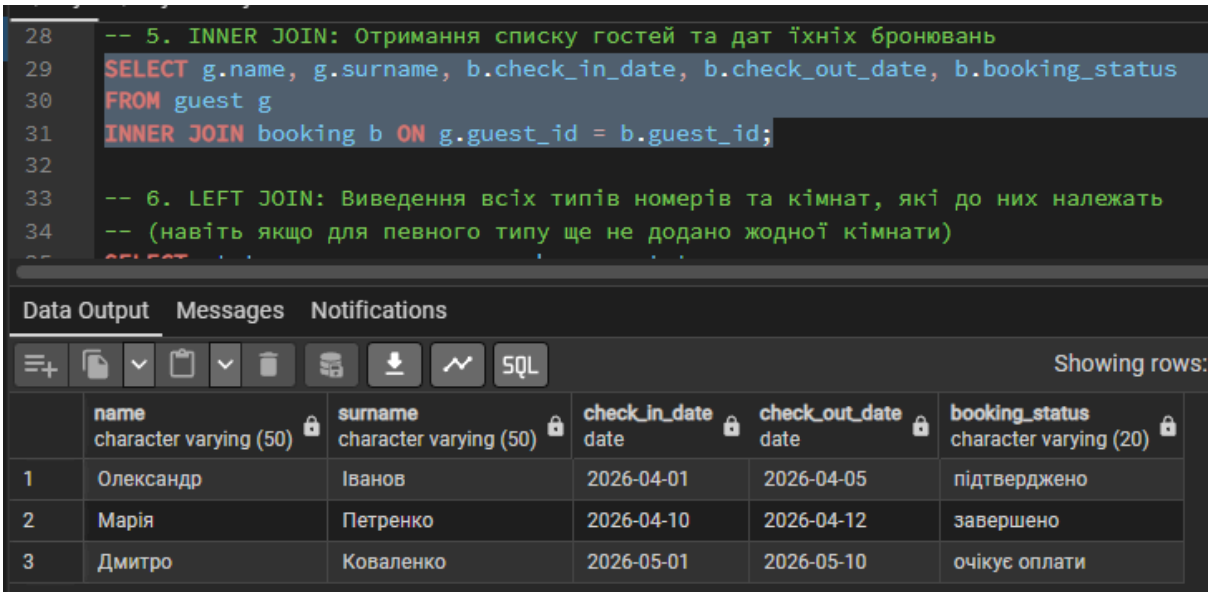
Запити з використанням JOIN (INNER JOIN, LEFT JOIN, RIGHT JOIN):

```
-- 5. INNER JOIN: Отримання списку гостей та дат їхніх бронювань
SELECT g.name, g.surname, b.check_in_date, b.check_out_date,
b.booking_status
FROM guest g
INNER JOIN booking b ON g.guest_id = b.guest_id;

-- 6. LEFT JOIN: Виведення всіх типів номерів та кімнат, які до них
належать
-- (навіть якщо для певного типу ще не додано жодної кімнати)
```

```
SELECT rt.type_name, r.room_number, r.status
FROM room_type rt
LEFT JOIN room r ON rt.type_id = r.type_id;

-- 7. RIGHT JOIN: Виведення всіх бронювань і прив'язаних до них платежів
-- (включаючи бронювання, які ще очікують оплати і не мають платежів)
SELECT b.booking_id, b.check_in_date, p.payment_id, p.amount
FROM payment p
RIGHT JOIN booking b ON p.booking_id = b.booking_id;
```



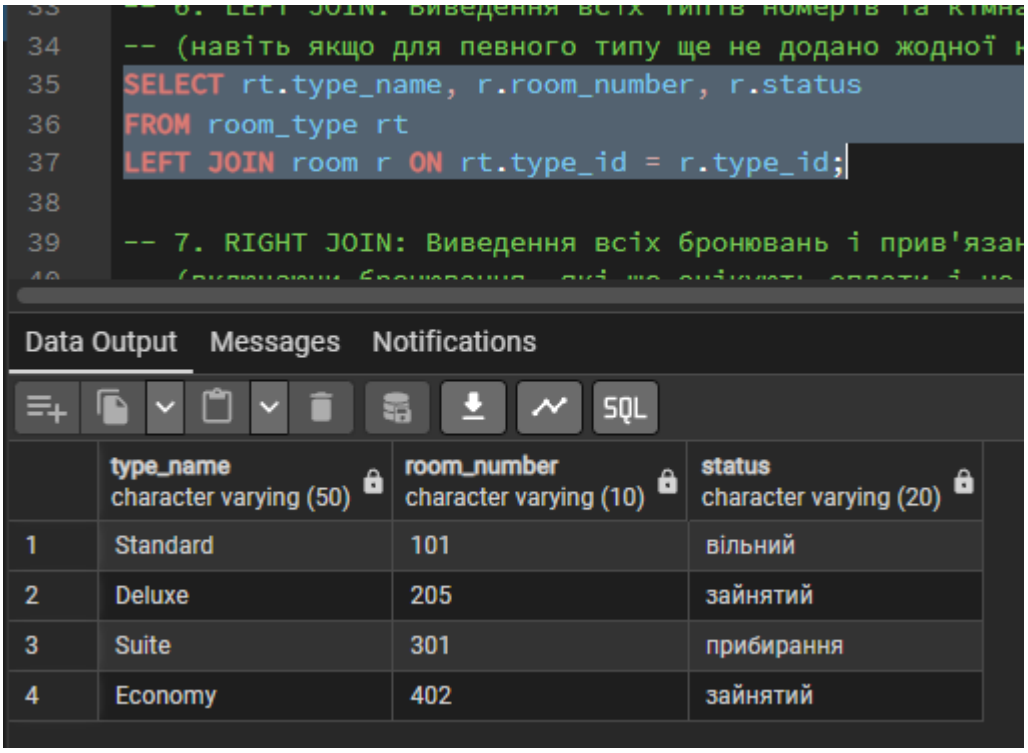
```
-- 5. INNER JOIN: Отримання списку гостей та дат їхніх бронювань
SELECT g.name, g.surname, b.check_in_date, b.check_out_date, b.booking_status
FROM guest g
INNER JOIN booking b ON g.guest_id = b.guest_id;

-- 6. LEFT JOIN: Виведення всіх типів номерів та кімнат, які до них належать
-- (навіть якщо для певного типу ще не додано жодної кімнати)
```

Data Output | Messages | Notifications

Showing rows:

	name character varying (50)	surname character varying (50)	check_in_date date	check_out_date date	booking_status character varying (20)
1	Олександр	Іванов	2026-04-01	2026-04-05	підтверджено
2	Марія	Петренко	2026-04-10	2026-04-12	завершено
3	Дмитро	Коваленко	2026-05-01	2026-05-10	очікує оплати



```
-- (навіть якщо для певного типу ще не додано жодної к
SELECT rt.type_name, r.room_number, r.status
FROM room_type rt
LEFT JOIN room r ON rt.type_id = r.type_id;

-- 7. RIGHT JOIN: Виведення всіх бронювань і прив'язан
```

Data Output | Messages | Notifications

	type_name character varying (50)	room_number character varying (10)	status character varying (20)
1	Standard	101	вільний
2	Deluxe	205	зайнятий
3	Suite	301	прибирання
4	Economy	402	зайнятий

```
-- 7. RIGHT JOIN: Виведення всіх бронювань і прив'язаних до н
-- (включаючи бронювання, які ще очікують оплати і не мають п
SELECT b.booking_id, b.check_in_date, p.payment_id, p.amount
FROM payment p
RIGHT JOIN booking b ON p.booking_id = b.booking_id;
```

Data Output Messages Notifications

	booking_id integer	check_in_date date	payment_id integer	amount numeric (8,2)
1	1	2026-04-01	1	4000.00
2	2	2026-04-10	2	5000.00
3	3	2026-05-01	3	1000.00

Запити з використанням підзапитів (SELECT, WHERE, HAVING):

```
-- 8. Підзапит у WHERE: Знайти платежі, сума яких перевищує середню
суму платежу по всьому готелю
SELECT payment_id, amount, payment_date
FROM payment
WHERE amount > (SELECT AVG(amount) FROM payment);

-- 9. Підзапит у SELECT: Вивести імена всіх гостей і кількість їхніх
бронювань окремою колонкою
SELECT
   name,
   surname,
   (SELECT COUNT(*) FROM booking b WHERE b.guest_id = g.guest_id) AS
total_bookings
FROM guest g;

-- 10. Підзапит у HAVING: Знайти типи номерів (ID), де кількість
наявних кімнат більша,
-- ніж середня кількість кімнат, що припадає на кожен тип.
SELECT type_id, COUNT(room_id) AS room_count
FROM room
GROUP BY type_id
HAVING COUNT(room_id) >= (
   SELECT AVG(cnt)
```

```
   FROM (SELECT COUNT(room_id) AS cnt FROM room GROUP BY type_id)
sub
);
```

```sql
SELECT payment_id, amount, payment_date
FROM payment
WHERE amount > (SELECT AVG(amount) FROM payment);

-- 9. Підзапит у SELECT: Вивести імена всіх гостей
SELECT
    name,
```

Data Output Messages Notifications

	payment_id [PK] integer	amount numeric (8,2)	payment_date timestamp without time zone
1	1	4000.00	2026-03-15 10:30:00
2	2	5000.00	2026-04-12 12:00:00

```sql
SELECT
    name,
    surname,
    (SELECT COUNT(*) FROM booking b WHERE b.guest_id = g.guest_id) AS total_bookings
FROM guest g;

-- 10. Підзапит у HAVING: Знайти типи номерів (ID), де кількість наявних кімнат більша
```

Data Output Messages Notifications

Showing rows: 1 to 3

	name character varying (50)	surname character varying (50)	total_bookings bigint
1	Олександр	Іванов	1
2	Марія	Петренко	1
3	Дмитро	Коваленко	1

```sql
SELECT type_id, COUNT(room_id) AS room_count
FROM room
GROUP BY type_id
HAVING COUNT(room_id) >= (
    SELECT AVG(cnt)
    FROM (SELECT COUNT(room_id) AS cnt FROM room GROUP BY type_id) sub
);
```

Data Output Messages Notifications

Show

	type_id integer	room_count bigint
1	3	1
2	4	1
3	2	1
4	1	1

<u>Висновок:</u> Під час виконання лабораторної роботи №4 було набуто практичних навичок написання та виконання аналітичних SQL-запитів (OLAP) у СУБД PostgreSQL на базі розробленої реляційної схеми «Готель». На практиці було закріплено використання агрегатних функцій (SUM, AVG, COUNT, MIN, MAX) спільно з операторами GROUP BY та HAVING для обчислення зведеної статистики та фільтрації згрупованих даних. Крім того, успішно реалізовано вибірку даних з кількох пов'язаних таблиць за допомогою різних типів об'єднань (INNER JOIN, LEFT JOIN, RIGHT JOIN) , а також побудовано складні багаторівневі запити з використанням підзапитів у конструкціях SELECT, WHERE та HAVING. Виконані завдання дозволили перетворити початкові "сирі" дані бази на змістовну аналітичну інформацію, яка є основою для формування звітів та підсумовування тенденцій.